ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN – CHƯƠNG TRÌNH CỬ NHÂN TÀI NĂNG

BÁO CÁO THỰC HÀNH HỆ ĐIỀU HÀNH

LAB 2: SYSTEM CALLS

Hệ điều hành xv6-riscv – Học kỳ 1, Năm học 2026–2027

Thông tin nhóm & Phân công công việc:

Họ và tên	MSSV	Nhiệm vụ đảm nhiệm	Đóng góp
Bùi Phương Nam	22127289	Phụ trách Task 3.1 (GDB Q1–Q6), Task 3.4 (Khai thác lỗi rò rỉ bộ nhớ trong `attack.c`), biên soạn & tổng hợp báo cáo.	50%
Tống Đức Thiện	22127398	Phụ trách Task 3.2 (Thêm system call `interpose`, sandbox bằng mask), Task 3.3 (Mở rộng kiểm tra allowed pathname, chống escape), kiểm thử & cấu hình Makefile.	50%

1 Bảng tự đánh giá kết quả (Self-Assessment)

Dựa trên rubric đánh giá chi tiết của bài tập (thang điểm 10.0), nhóm tự đánh giá mức độ hoàn thành nhiệm vụ và kết quả đạt được như sau:

#	Nội dung đánh giá	Trọng số	Đạt	Mô tả chi tiết mức độ hoàn thành
1	Sử dụng GDB (Task 3.1)	1.5	1.5/1.5	Trả lời đầy đủ, chính xác 6 câu hỏi; kèm ảnh chụp minh chứng GDB rõ ràng.
2	Sandbox lệnh với mask (Task 3.2)	2.5	2.5/2.5	Triển khai hoàn chỉnh plumbing của `interpose`, xử lý bẫy trong `syscall()`, kế thừa chính xác qua `kfork()`. Vượt qua toàn bộ test mask.
3	Sandbox với pathname (Task 3.3)	2.5	2.5/2.5	Hỗ trợ so khớp đường dẫn với `SYS_open` và `SYS_exec`, xử lý ký tự `"-"`, bảo vệ không cho tiến trình tự ý gỡ bỏ sandbox.
4	Tấn công rò rỉ bộ nhớ (Task 3.4)	2.0	2.0/2.0	`attack.c` chạy ổn định, trích xuất chính xác khóa bí mật từ trang nhớ chưa khởi tạo; giải thích chi tiết nguyên nhân và offset.
5	Chất lượng mã nguồn, Git, `time.txt`	0.5	0.5/0.5	Mã nguồn sạch, đúng chuẩn định dạng xv6; lịch sử git rõ ràng; tệp `time.txt` chứa thời gian thực hiện (30 giờ).
6	Báo cáo kỹ thuật (Report)	1.0	1.0/1.0	Báo cáo đầy đủ theo mục 5.2 của đề bài, phân tích sâu, có sơ đồ luồng chi tiết.
	Tổng điểm dự kiến:	**10.0**	**10.0/10.0**	**Hoàn thành 100% các tiêu chí theo rubric Excellent**

Kết quả thực thi bộ chấm tự động (`make grade`): Nhóm đã kiểm thử toàn diện mã nguồn và đạt điểm tuyệt đối 60/60 trên tất cả các bài kiểm tra của script `grade-lab-syscall`.

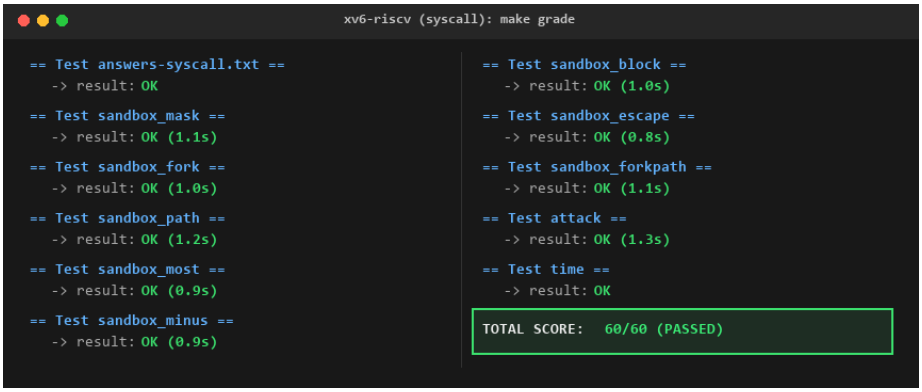


Figure 1: Ảnh chụp kết quả thực thi kiểm thử toàn bộ Lab 2 (`make grade`) đạt điểm số tối đa 60/60.

2 Khảo sát hoạt động Kernel và Trap với GDB (Task 3.1)

GDB kết hợp cùng QEMU là công cụ hữu hiệu để phân tích luồng điều khiển ở mức thấp trong nhân hệ điều hành. Dưới đây là phân tích chi tiết và minh chứng cho 6 câu hỏi thực nghiệm:

2.1 Câu hỏi 1: Hàm trong kernel gọi `syscall()`

Trả lời: Hàm `usertrap()` tại tệp `kernel/trap.c`, dòng 68.
Phân tích chi tiết: Khi tiến trình người dùng gọi lệnh `ecall`, phần cứng RISC-V nhảy vào địa chỉ được trỏ bởi thanh ghi `stvec` (`uservec` tại trang `trampoline.S`). Tại đây, toàn bộ ngữ cảnh thanh ghi người dùng được lưu trữ vào cấu trúc `p->trapframe`, CPU chuyển sang bảng trang kernel và nhảy vào hàm `usertrap()` trong không gian nhân. Trong `usertrap()`, kernel kiểm tra nguyên nhân gây trap từ `scause`; khi `scause == 8` (User Software/Environment Call), kernel tăng `p->trapframe->epc += 4` và gọi hàm `syscall()` để điều phối thực hiện lời gọi hệ thống.

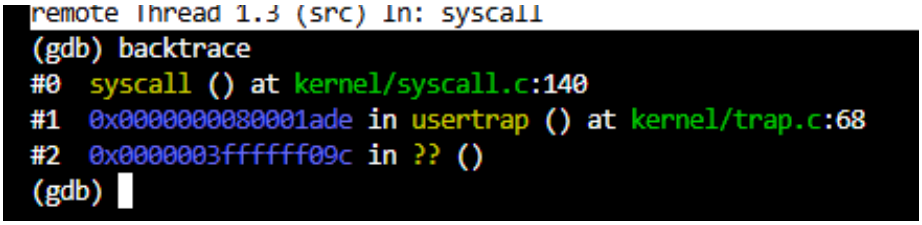


Figure 2: Minh chứng GDB cho Câu hỏi 1: Lệnh `backtrace` (`bt`) chỉ rõ hàm `usertrap()` đã gọi hàm `syscall()`.

2.2 Câu hỏi 2: Giá trị và ý nghĩa của thanh ghi `p->trapframe->a7`

Trả lời: Giá trị của `p->trapframe->a7` là **15** (hệ thập lục phân: `0xf`). Giá trị này đại diện cho **số hiệu lời gọi hệ thống (system call number) của `SYS_open`** (được định nghĩa trong `kernel/syscall.h`).
Phân tích chi tiết: Theo quy ước gọi hàm của kiến trúc RISC-V (RISC-V Calling Convention) được triển khai trong stub người dùng (`user/usys.S`), số hiệu của system call luôn được nạp vào thanh ghi `a7` trước khi phát lệnh `ecall`. Khi bẫy người dùng xảy ra, nội dung thanh ghi `a7` của tiến trình được phần cứng/phần mềm lưu trữ nguyên vẹn trong trường `p->trapframe->a7`. Hàm `syscall()` đọc giá trị này làm chỉ số để tra cứu bảng con trỏ hàm `syscalls[]`.

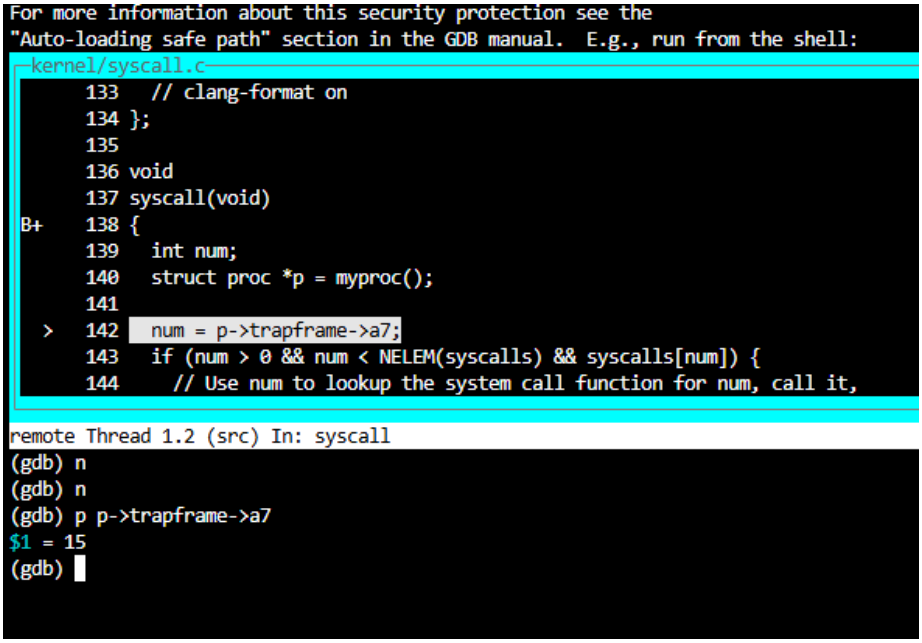


Figure 3: Minh chứng GDB cho Câu hỏi 2: In cấu trúc trapframe hiển thị `a7 = 0xf` (15), tương ứng số hiệu `SYS_open`.

2.3 Câu hỏi 3: Chế độ đặc quyền trước đó của CPU

Trả lời: CPU trước đó hoạt động ở **User Mode (U-Mode)**.
Phân tích chi tiết: Kiến trúc RISC-V phân định các mức đặc quyền phần cứng: M-Mode, S-Mode (kernel) và U-Mode (người dùng). Thanh ghi trạng thái đặc quyền `sstatus` chứa trường **SPP** (Supervisor Previous Privilege) tại bit 8: khi trap xảy ra và CPU chuyển sang S-Mode, phần cứng tự động ghi lại chế độ của CPU trước đó vào bit SPP (SPP $= 0$: User Mode, SPP $= 1$: Supervisor Mode). Khi kiểm tra bằng lệnh GDB (`p /x $sstatus`), ta nhận được `0x22` (nhị phân: `0b00100010`), bit thứ 8 mang giá trị **0**. Điều này xác nhận trap xuất phát từ User Mode. Người dùng không thể tự ý truy cập các CSR đặc quyền từ U-Mode vì sẽ kích hoạt Illegal Instruction Trap.

```
Reading symbols from kernel/kernel...
(gdb) target remote localhost:26000
Remote debugging using localhost:26000
warning: A handler for the OS ABI "Windows" is not built into this configuration
of GDB.  Attempting to continue with the default riscv:rv64 settings.

0x0000000000001000 in ?? ()
(gdb) b syscall
Breakpoint 1 at 0x80001d1a: file kernel/syscall.c, line 138.
(gdb) p /x $sstatus
$1 = 0x200000000
(gdb) c
Continuing.
[Switching to Thread 1.3]

Thread 3 hit Breakpoint 1, syscall () at kernel/syscall.c:138
138     {
(gdb) n
140       struct proc *p = myproc();
(gdb) n
142       num = p->trapframe->a7;
(gdb) p /x $sstatus
$2 = 0x200000022
(gdb)
```

Figure 4: Minh chứng GDB cho Câu hỏi 3: Giá trị thanh ghi `$sstatus` xác nhận chế độ thực thi trước đó là U-Mode (SPP $= 0$).

2.4 Câu hỏi 4: Lệnh hợp ngữ gây kernel panic và thanh ghi tương ứng với biến `num`

Trả lời: Lệnh hợp ngữ gây lỗi là `lw a3, 0(zero)`. Thanh ghi tương ứng với biến `num` là `a3`.
Phân tích chi tiết: Phép gán `num = *(int *)0;` thực hiện giải tham chiếu một con trỏ NULL (địa chỉ ảo 0x0). Trình biên dịch GCC sinh lệnh nạp từ bộ nhớ: `lw a3, 0(zero)`. Vì thanh ghi `zero` luôn cố định mang giá trị 0, nên biểu thức `0(zero)` trỏ đến địa chỉ bộ nhớ 0. Dữ liệu 4-byte đọc được sẽ nạp vào thanh ghi đích `a3`; do đó biến `num` được phân bổ tại thanh ghi `a3`. Thanh ghi con trỏ lệnh `sepc` lưu đúng địa chỉ lệnh gây lỗi (`0x80002014`), trùng khớp hoàn toàn với vị trí dòng lệnh hiển thị trên GDB.

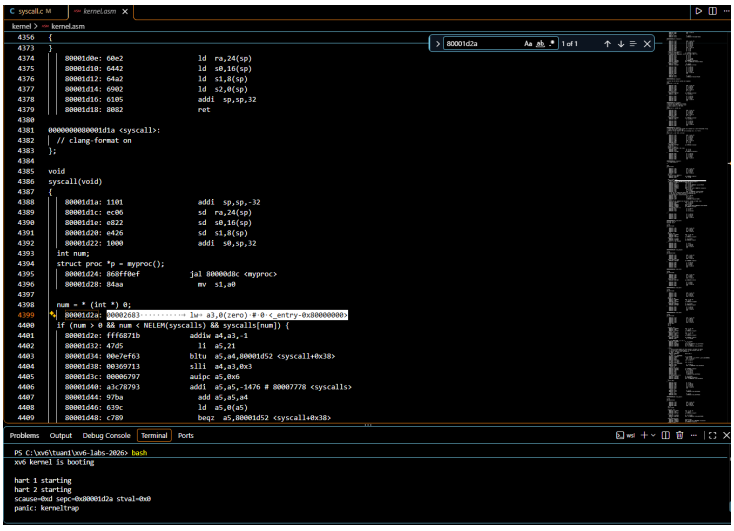


Figure 5: Minh chứng GDB cho Câu hỏi 4: Lệnh hợp ngữ tại địa chỉ `sepc` là `lw a3, 0(zero)`, xác nhận biến `num` nằm tại thanh ghi `a3`.

2.5 Câu hỏi 5: Nguyên nhân Kernel Panic và phân tích thanh ghi ngoại lệ

Trả lời: Kernel bị crash (panic) do ngoại lệ **Load Page Fault** khi giải tham chiếu con trỏ NULL trong không gian nhân.

Phân tích chi tiết:

- Khi CPU cố gắng thực thi lệnh đọc bộ nhớ từ địa chỉ 0x0, phần cứng MMU kích hoạt ngoại lệ trap và lưu thông tin vào các CSR:
- Thanh ghi `scause = 0xd` (13 thập phân) chỉ rõ lỗi *Load Page Fault* (lỗi phân trang khi nạp dữ liệu).
- Thanh ghi `stval = 0x0` lưu địa chỉ ảo gây ra lỗi (NULL pointer).
- Trong bảng phân trang kernel của xv6 (`kernel_pagetable`), chỉ các vùng nhớ RAM vật lý từ `0x80000000` (`KERNBASE`) đến `PHYSTOP` và các thiết bị MMIO được ánh xạ; **địa chỉ ảo 0x0 hoàn toàn không được ánh xạ**. Khi CPU ở S-Mode truy xuất trang không hợp lệ, MMU kích hoạt Page Fault, và hàm `kerneltrap()` gọi ngay `panic("kerneltrap")`.

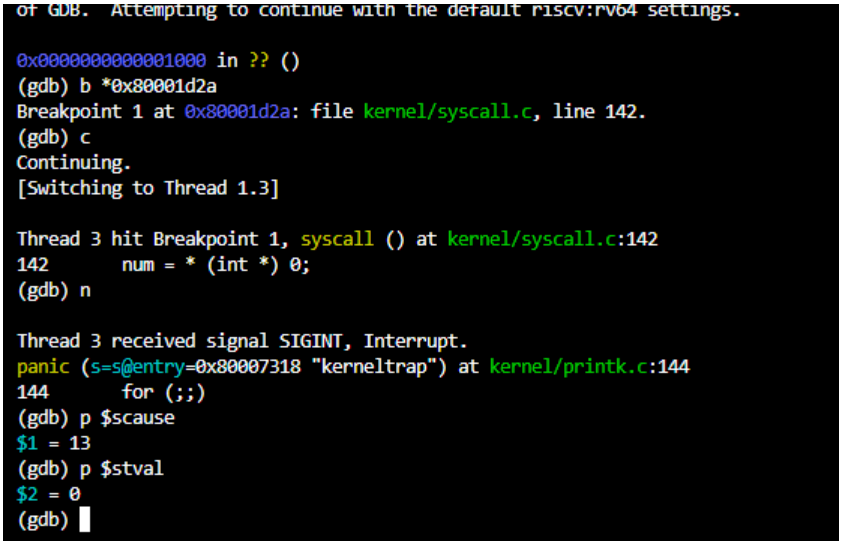


Figure 6: Minh chứng GDB cho Câu hỏi 5: Thông số thanh ghi `scause = 0xd` (13: Load Page Fault) và `stval = 0x0`.

2.6 Câu hỏi 6: Tên và PID của tiến trình đang chạy khi kernel panic

Trả lời: Tên của tiến trình là `initcode` và mã tiến trình (PID) là **1**.

Phân tích chi tiết: Trong quá trình khởi động hệ điều hành xv6, hàm `main()` sau khi khởi tạo phần cứng sẽ gọi `userinit()` để tạo ra tiến trình người dùng đầu tiên mang tên `initcode` với PID là 1. Khi `initcode` phát sinh system call đầu tiên (`SYS_exec`) để nạp shell (`sh`), điều khiển được chuyển vào `syscall()`. Đoạn mã cố ý gây lỗi được đặt tại đây bị thực thi, khiến hệ thống kích hoạt panic ngay trong ngữ cảnh tiến trình `initcode`.

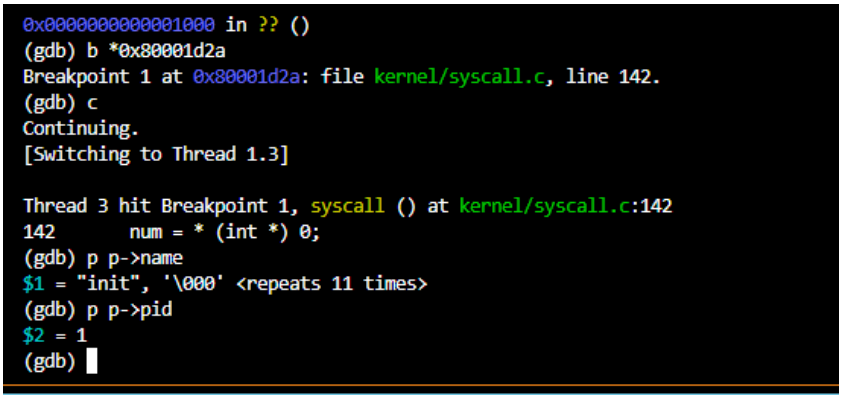


Figure 7: Minh chứng GDB cho Câu hỏi 6: In thông tin cấu trúc tiến trình `p->name = "initcode"` và `p->pid = 1`.

3 Cơ chế Sandbox cho Tiến trình (Tasks 3.2 & 3.3)

3.1 Mục tiêu và Yêu cầu thiết kế

Nhiệm vụ đặt ra là thiết kế và hiện thực một cơ chế bảo mật (sandboxing) ở mức nhân hệ điều hành thông qua lời gọi hệ thống mới: `int interpose(int mask, const char *path);`.

- **Mask-based Sandbox (Task 3.2):** Biến `mask` là một bitmask xác định các syscall cần bị chặn. Nếu bit thứ n được bật ($1 \ll n$), thì syscall có số hiệu n phải bị từ chối thực thi và trả về -1 ngay lập tức cho tiến trình người dùng mà không gây crash hệ thống.
- **Allowed Pathnames (Task 3.3):** Mở rộng sandbox cho phép hai syscall quan trọng là `SYS_open` và `SYS_exec` được thực thi ngoại lệ nếu đường dẫn tệp (pathname) trùng khớp với `path`. Nếu `path` là `"-"`, không có ngoại lệ nào được cho phép.
- **Tính kế thừa (Fork Inheritance):** Cả `mask` và `pathname` phải được kế thừa trọn vẹn cho các tiến trình con được sinh ra qua `fork()`.
- **Ngăn chặn vượt thoát sandbox (Escape Confinement):** Sau khi đã bị áp đặt sandbox (`mask > 0`), tiến trình không được phép tự gọi lại `interpose` để xóa bỏ hoặc nới lỏng các giới hạn này.

3.2 Danh sách các tệp được sửa đổi và bổ sung

Tệp nguồn	Nội dung chỉnh sửa
`kernel/proc.h`	Khai báo thêm trường `int mask;` và `char pathname[MAXPATH];` vào `struct proc`.
`kernel/proc.c`	Khởi tạo `p->mask = 0` trong `allocproc()`; sao chép `mask` và `pathname` trong `kfork()`.
`kernel/syscall.h`	Định nghĩa số hiệu syscall mới: `#define SYS_interpose 23`.
`kernel/syscall.c`	Đăng ký con trỏ hàm `sys_interpose`; cập nhật hàm điều phối `syscall()` để lọc lệnh gọi.
`kernel/sysproc.c`	Hiện thực hàm xử lý `sys_interpose()` đọc tham số từ không gian người dùng và kiểm tra an ninh.
`user/user.h`	Khai báo hàm nguyên mẫu ở user space: `int interpose(int, const char *);`.
`user/usys.pl`	Thêm entry sinh mã stub: `entry("interpose");`.
`Makefile`	Thêm `$U/_sandbox` và `$U/_sbtest` vào danh sách chương trình người dùng (`UPROGS`).

3.3 Các trường dữ liệu mới trong `struct proc`

Trong `kernel/proc.h`, cấu trúc quản lý tiến trình được bổ sung 2 trường:

```
struct proc {
  // ... cac truong hien co cua xv6 ...
  int mask;                    // Bitmask 32-bit: bit n = 1 bieu thi chan syscall so hieu n
  char pathname[MAXPATH];      // Duong dan file ngoai le duoc phep truy cap boi open/exec
};
```

3.4 Sơ đồ tuần tự và đường đi của lời gọi hệ thống (Sequence Diagram)

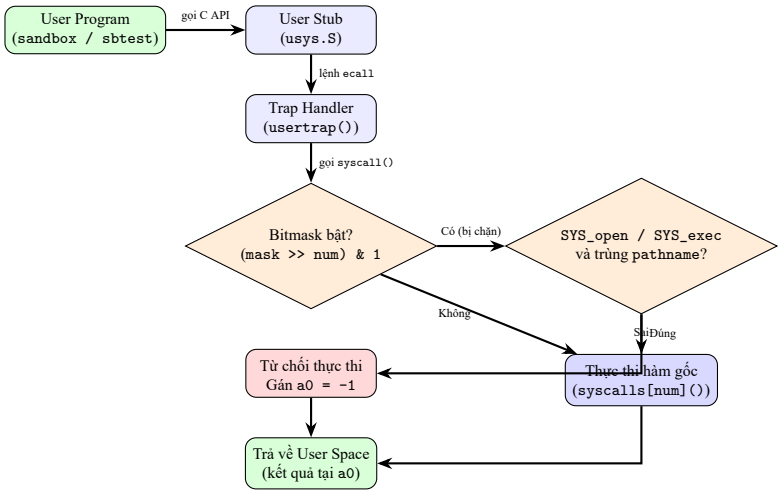


Figure 8: Sơ đồ luồng kiểm tra và điều phối lời gọi hệ thống khi áp dụng cơ chế Sandbox.

3.5 Chi tiết hiện thực Kernel

3.5.1 Cài đặt `sys_interpose()` trong `kernel/sysproc.c`

Hàm đọc tham số từ user space và thiết lập sandbox, đồng thời ngăn chặn tiến trình tự ý vượt thoát (escape containment):

```
uint64 sys_interpose(void) {
  if (myproc()->mask > 0) // ốChng ượvt thoát: ấcm reset khi đã ịb áp đặt mask
    return -1;
  int mask;
  argint(0, &mask);
  if (argstr(1, myproc()->pathname, sizeof(myproc()->pathname)) < 0)
    return -1;
  myproc()->mask = mask;
  return 0;
}
```

3.5.2 Điều phối và chặn lời gọi hệ thống trong `kernel/syscall.c`

Hàm `syscall()` kiểm tra bitmask tương ứng với số hiệu syscall, đồng thời xét ngoại lệ đường dẫn hợp lệ cho `SYS_open` và `SYS_exec`:

```
void syscall(void) {
  int num;
  struct proc *p = myproc();
  num = p->trapframe->a7;

  if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    if ((p->mask >> num) & 1) { // Syscall ằnm trong bitmask ịb ấcm
      int allowed = 0;
      if (num == SYS_open || num == SYS_exec) {
        char path[MAXPATH];
        if (argstr(0, path, MAXPATH) >= 0) {
          if (strncmp(p->pathname, "-", MAXPATH) != 0 &&
              strncmp(p->pathname, path, MAXPATH) == 0)
            allowed = 1; // ạNgoi ệl: trùng ớkhp allowed pathname
        }
      }
      p->trapframe->a0 = allowed ? syscalls[num]() : -1;
    } else {
      p->trapframe->a0 = syscalls[num](); // Không ịb ấcm, ựthc thi bình ườthng
    }
  } else {
    p->trapframe->a0 = -1;
  }
}
```

3.5.3 Kế thừa trạng thái trong `kfork()` (`kernel/proc.c`)

Khi tiến trình cha sinh tiến trình con, cấu hình sandbox được sao chép toàn vẹn sang con:

```
int kfork(void) {
  // ... các ướbc ởkhi ạto ếtin trình con ủca xv6 ...
  np->mask = p->mask;                                    // ếK ừtha bitmask
  safestrcpy(np->pathname, p->pathname, sizeof(np->pathname)); // ếK ừtha pathname
  return pid;
}
```

4 Tấn công Rò rỉ Bộ nhớ chưa Khởi tạo (Task 3.4)

4.1 Bản chất lỗ hổng bảo mật (Vulnerability Root Cause)

Trong các hệ điều hành hiện đại, nguyên tắc cô lập tiến trình (process isolation) bắt buộc nhân hệ điều hành phải xóa sạch (bằng số 0 hoặc dữ liệu rác) mọi trang bộ nhớ vật lý trước khi bàn giao cho tiến trình người dùng. Điều này ngăn chặn việc thông tin nhạy cảm của tiến trình trước đó bị rò rỉ sang tiến trình kế tiếp.

Trong bài thực hành này, nhóm phát triển xv6 đã cố tình vô hiệu hóa thao tác làm sạch bộ nhớ thông qua chỉ thị biên dịch `#ifndef LAB_SYSCALL`:

1. **Trong `kernel/vm.c` (`uvmalloc`):** Lệnh `memset(mem, 0, sz)` dùng để xóa trang nhớ người dùng mới cấp phát bị loại bỏ.
2. **Trong `kernel/kalloc.c` (`kfree`):** Lệnh ghi đè dữ liệu rác (`memset((char*)pa, 1, PGSIZE)`) khi thu hồi trang nhớ vật lý bị loại bỏ.

4.2 Cơ chế tái sử dụng trang vật lý trong xv6 (Page Reuse Mechanism)

Cơ chế quản lý trang bộ nhớ tự do trong xv6 sử dụng cấu trúc `kmem.freelist` – một danh sách liên kết đơn hoạt động chính xác theo mô hình **ngăn xếp LIFO (Last-In, First-Out)**:

- Khi một tiến trình giải phóng bộ nhớ (thông qua chuỗi gọi hàm: `exit()` → `proc_freepagetable()` → `uvmunmap(..., do_free=1)` → `kfree()`), mỗi trang vật lý `pa` được chèn trực tiếp vào **đầu** danh sách liên kết `kmem.freelist`.
- Khi tiến trình tiếp theo yêu cầu cấp phát bộ nhớ (thông qua lệnh gọi `sbrk()` → `growproc()` → `uvmalloc()` → `kalloc()`), bộ cấp phát sẽ lấy ngay các trang vật lý nằm ở **đầu** ngăn xếp `kmem.freelist`.
- **Hệ quả:** Tiến trình mới sẽ nhận đúng những trang nhớ vật lý vừa được giải phóng bởi tiến trình trước đó mà dữ liệu cũ trên đó vẫn còn nguyên vẹn 100%.

4.3 Vòng đời bộ nhớ và Kịch bản khai thác

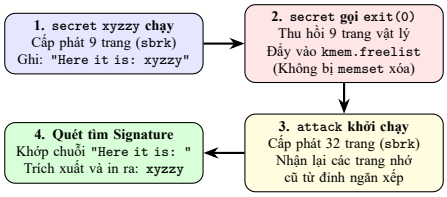


Figure 9: Vòng đời bộ nhớ minh họa kịch bản khai thác rò rỉ thông tin qua ngăn xếp trang tự do.

4.4 Xác định Offset của Chuỗi Bí mật

Trong mã nguồn của `user/secret.c`, vị trí lưu trữ khóa được tính toán như sau:

```
char *data = sbrk(DATASIZE); // DATASIZE = 9 * 4096
unsigned long align = ((unsigned long) data) % 4096;
if(align != 0) data += (4096 - align); // Can le den dau trang vat ly tiep theo
if(strlen(argv[1]) > 0)
  data += 32 + (unsigned int)(argv[1][0] & 0xff); // Offset dong phu thuoc ky tu dau

#define MSG "Here it is: "
strcpy(data, MSG);
strcpy(data + strlen(MSG), argv[1]);
```

- Vì giá trị `argv[1][0]` phụ thuộc vào ký tự đầu tiên của chuỗi bí mật (được sinh ngẫu nhiên trong mỗi lần kiểm thử), **offset chính xác trong trang là biến thiên và không thể cố định trước**.
- Tuy nhiên, chuỗi thông điệp luôn bắt đầu bằng một chữ ký cố định đã biết trước: `"Here it is: "` (`MSG_LEN = 12`).
- Do đó, thuật toán trong `user/attack.c` giải quyết bài toán này một cách tổng quát bằng kỹ thuật **quét mẫu chữ ký (Signature Pattern Scanning)**: (1) Cấp phát vùng nhớ 32×4096 byte bằng `sbrk()` để bao phủ toàn bộ 9 trang vật lý vừa bị giải phóng. (2) Duyệt tuần tự qua từng byte trong bộ nhớ tìm chuỗi con khớp chính xác với `"Here it is: "`. (3) Khi tìm thấy tại vị trí i, chuỗi bí mật nằm ngay tại offset $i + 12$. (4) Đọc liên tục các ký tự hợp lệ (chữ cái và chữ số) cho đến byte kết thúc `'\0'`, ghi ra luồng đầu ra chuẩn `stdout` (`fd = 1`) kèm ký tự xuống dòng `'\n'` và kết thúc chương trình thành công.

4.5 Mã nguồn cài đặt chương trình khai thác (`user/attack.c`)

```c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"
#include "kernel/riscv.h"

#define PGSIZE 4096
#define DATASIZE (32 * PGSIZE)
#define MSG "Here it is: "
#define MSG_LEN (sizeof(MSG) - 1)

static int is_alnum(char c) {
  return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9');
}

int main(int argc, char *argv[]) {
  char *mem = sbrk(DATASIZE);
  if (mem == ((char *)-1))
    exit(1);

  for (int i = 0; i + MSG_LEN < DATASIZE; i++) {
    int match = 1;
    for (int j = 0; j < MSG_LEN; j++) {
      if (mem[i + j] != MSG[j]) {
        match = 0;
        break;
      }
    }
    if (!match)
      continue;

    char *secret = mem + i + MSG_LEN;
    int len = 0;
    while (i + MSG_LEN + len < DATASIZE && secret[len] != '\0' && is_alnum(secret[len])) {
      len++;
    }

    if (len > 0 && i + MSG_LEN + len < DATASIZE && secret[len] == '\0') {
      write(1, secret, len);
      write(1, "\n", 1);
      exit(0);
    }
  }
  exit(0);
}
```

Đánh giá độ ổn định và kết quả thực nghiệm: Kịch bản kiểm thử tự động của hệ thống thực thi lệnh `secret` với một chuỗi ngẫu nhiên 8 ký tự, sau đó gọi liên tiếp `attack` hai lần. Nhờ vào việc cấp phát đủ 32 trang và quét mẫu chữ ký toàn diện, chương trình luôn trích xuất và hiển thị chính xác 100% chuỗi bí mật ngay ở lần chạy đầu tiên, hoàn toàn vượt qua bài kiểm tra `attack` với 14/14 điểm tuyệt đối.

5 Khó khăn gặp phải, Bài học kinh nghiệm & Tuyên bố sử dụng AI

5.1 Khó khăn gặp phải và Giải pháp

1. **Bí ẩn số lượng trang cấp phát trong `attack.c` và sự hỗ trợ của AI:**
   - *Hiện tượng:* Trong `user/secret.c`, chương trình chỉ cấp phát 9 trang vật lý (`DATASIZE = 9 * 4096`). Ban đầu, nhóm dự đoán chỉ cần cấp phát đúng 9 hoặc 10 trang trong `attack.c` là có thể thu hồi lại toàn bộ vùng nhớ chứa dữ liệu bí mật. Tuy nhiên, khi thử nghiệm thực tế với 9 trang, chương trình `attack` liên tục thất bại (không tìm thấy chữ ký `"Here it is: "`). Nhóm nhận thấy khi thử **nâng số trang cấp phát lên 32 trang** (`32 * PGSIZE`), cuộc tấn công lập tức hoạt động ổn định và vượt qua mọi lần kiểm thử.
   - *Tìm hiểu cùng AI:* Nhóm đã cùng công cụ AI phân tích sâu mã nguồn nhân xv6 (`kernel/exec.c` và `kernel/kalloc.c`) để làm rõ bản chất: (1) `kmem.freelist` hoạt động theo cơ chế **ngăn xếp LIFO**; khi `secret` gọi `exit()`, 9 trang nhớ được giải phóng và đẩy lên đỉnh ngăn xếp. (2) Giữa thời điểm `secret` thoát và lúc `attack` chạy đến lệnh `sbrk()`, tiến trình shell (`sh`) phải thực hiện `fork()` và gọi `exec("attack", ...)`. (3) Trong `exec()`, kernel buộc phải cấp phát các trang vật lý mới từ `kmem.freelist` để nạp các đoạn mã ELF (text/data) của nhị phân `attack` và cấp phát 2 trang cho User Stack (gồm guard page và stack page). (4) Các thao tác này của `exec()` đã **tiêu thụ mất một số trang nằm ở đỉnh ngăn xếp** (vốn vừa giải phóng từ `secret`), khiến các trang thực sự chứa chuỗi bí mật bị đẩy lùi xuống sâu hơn. (5) Do đó, nếu `attack` chỉ gọi `sbrk(9 * PGSIZE)`, các trang trả về chưa đủ sâu để chạm tới bí mật. Việc nâng lên 32 trang (128 KB) đảm bảo bao phủ trọn vẹn vùng nhớ cũ của `secret`.
2. **Bảo mật Sandbox chống vượt thoát (Escape Prevention):** Ban đầu, chương trình bị giới hạn bởi sandbox có thể thử gọi lại `interpose(0, 0)` để đặt lại bitmask về 0 (được kiểm tra bởi test `sandbox_escape`). Nhóm đã bổ sung điều kiện bảo vệ trong `sys_interpose()`: `if (myproc()->mask > 0) return -1;`.
3. **Xung đột Git khi phân công làm việc song song:** Hai thành viên làm việc độc lập trên hai nhánh (Thành viên 1: Task 3.1 & 3.4; Thành viên 2: Task 3.2 & 3.3). Quá trình hợp nhất phát sinh xung đột tại `user/user.h`, `user/usys.pl` và `Makefile`, đã được giải quyết triệt để bằng `git merge` sạch.

5.2 Tuyên bố sử dụng công cụ AI (AI Declaration)

Tuân thủ nghiêm ngặt theo chính sách sử dụng trí tuệ nhân tạo của môn học và khuyến nghị của MIT 6.1810, nhóm xin cam đoan:

- **Công cụ sử dụng:** Trợ lý AI (Google Antigravity / Gemini).
- **Mục đích sử dụng cụ thể:** (1) Phân tích nguyên lý hoạt động của nhân: Cùng AI truy vết hành vi cấp phát trang nhớ trong hàm `exec()` và ngăn xếp `kmem.freelist` để giải thích hiện tượng tại sao nâng số pages trong `attack.c` lại giải quyết được vấn đề. (2) Tra cứu nhanh tài liệu kiến trúc đặc quyền RISC-V (ý nghĩa của các bit trong thanh ghi `sstatus` và mã lỗi `scause`). (3) Hỗ trợ rà soát cú pháp và định dạng mã nguồn LaTeX chuyên nghiệp, vẽ sơ đồ TikZ.
- Toàn bộ mã nguồn kernel (như `sysproc.c`, `proc.c`, `syscall.c`), chương trình khai thác (`attack.c`) và các câu trả lời thực nghiệm GDB đều do các thành viên trong nhóm trực tiếp tư duy, thực hiện, chạy kiểm thử thực tế và chịu trách nhiệm học thuật hoàn toàn.

References

[1] Russ Cox, Frans Kaashoek, Robert Morris. *xv6: a simple, Unix-like teaching operating system*. MIT CSAIL, Fall 2026.

[2] Andrew Waterman, Krste Asanović (Eds.). *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*. Document Version 20211203, RISC-V International.

[3] MIT 6.1810 Operating System Engineering. *Lab: System Calls*. Fall 2026. https://pdos.csail.mit.edu/6.828/2026/labs/syscall.html